LLM Gateway V1 — Project Overview

One-page-at-a-glance summary · unified multi-provider gateway (FastAPI + PostgreSQL + React + Docker)

1. Overview

What the project is, in one breath.

The **LLM Gateway** is a single backend API that sits in front of multiple LLM providers (Gemini and Groq today, extensible to others). Instead of every application integrating each provider separately, clients call one endpoint — **POST /v1/chat/completions** — and the gateway handles authentication, request validation, provider selection, calling the provider, normalizing the response, and logging usage. It ships as a three-service Docker stack (frontend, backend, PostgreSQL).

The core idea: one API, many providers, with cross-cutting concerns (auth, logging, error handling, secrets) centralized in one place rather than duplicated in every client.

2. Architecture

How a request travels through the system.

```
Browser
  -> React frontend (nginx)        [http://localhost:3000]
  -> FastAPI gateway               [http://localhost:8000]
       request-id middleware
       authentication (gateway API key)
       request validation (Pydantic)
       model registry  (alias -> provider + real model)
       GatewayService  (orchestration, no provider branching)
       provider factory -> Gemini / Groq adapter -> provider API
       response normalization (one common shape)
       usage logging
  -> PostgreSQL                    [postgres:5432]
```

The backend stays **provider-agnostic**: adding a provider means adding an adapter and a registry entry, with no changes to routes, GatewayService, or schemas.

3. Request flow (chat completion)

- A request-ID middleware assigns one **req_...** id and returns it as **X-Request-ID**.
- The route authenticates the gateway API key and opens a DB session (FastAPI dependencies).
- Pydantic validates the body; malformed requests get a 422 before any logic runs.
- GatewayService resolves the public model id via the registry and gets the matching adapter.
- The adapter translates to the provider's format, calls it over httpx (with a timeout), and normalizes the reply.
- The route logs usage metadata to PostgreSQL and returns the standardized response.

4. What's implemented (V1)

- Unified chat endpoint across providers, plus **/health** and **/v1/models**.
- Provider abstraction (ABC + adapter pattern); **Gemini** and **Groq** adapters.
- Model registry mapping public aliases to provider + real model id (config-driven).
- API-key authentication; keys stored only as **SHA-256 hashes** in PostgreSQL.

- Usage logging of per-request metadata (never prompts or responses).
- Request IDs correlated across response header, logs, and the database.
- Standardized error envelope with safe messages; configurable upstream timeouts.
- React console UI and a one-command Docker Compose stack.

5. Technology stack

Layer	Technology
Backend	Python 3.12, FastAPI, Uvicorn, Pydantic / Pydantic Settings
LLM calls	httpx (async) to Gemini & Groq REST APIs
Database	PostgreSQL 16, SQLAlchemy 2.0, Alembic, psycopg v3
Frontend	React + Vite, served by nginx
Infra	Docker Compose (frontend, backend, postgres)
Testing	pytest (74 tests)

6. Data model (PostgreSQL)

Table	Key columns	Purpose
users	id, name, created_at	The client/account behind a key
api_keys	id, user_id, key_hash, is_active, created_at	Gateway keys (hashed)
requests	request_id, user_id, api_key_id, model, provider, tokens, latency_ms, status, error_code	Usage metadata log

Privacy by default: only metadata is stored — there is no column for prompt or response content.

7. Security model — two kinds of API keys

	Gateway API key	Provider API key
Used by	client -> this gateway	the gateway -> the LLM provider
Sent as	Authorization: Bearer gw_...	provider-specific header (backend only)
Stored as	SHA-256 hash in PostgreSQL	backend .env only; never in the frontend

Raw keys are never stored or logged; provider failures are translated into safe errors so provider internals never reach the client.

8. Reliability & error handling

- Centralized exception handlers produce one JSON shape: **{"error":{"code","message"}}**.
- Status/code mapping: 401 UNAUTHORIZED, 404 UNKNOWN_MODEL, 422 VALIDATION_ERROR, 502 PROVIDER_ERROR, 503 PROVIDER_UNAVAILABLE, 500 INTERNAL_ERROR.
- Upstream calls are bounded by **LLM_TIMEOUT_SECONDS** (default 30); a timeout returns a clean 502.

- Usage logging is failure-safe — a DB logging error never changes the client's result.
- No automatic retries or provider fallback in V1 (deliberate scope choice).

9. Running it

```
# 1. Put provider keys + GATEWAY_API_KEYS in backend/.env (from .env.example)
# 2. Start the whole stack:
docker compose up --build

# 3. Seed a gateway API key (once):
docker compose exec backend python -m app.db.seed

# 4. Run the test suite inside the container:
docker compose exec backend pytest -q

# Frontend: http://localhost:3000   Backend/docs: http://localhost:8000/docs
```

The backend runs **alembic upgrade head** before serving; a failed migration stops the container instead of serving a broken schema.

10. Scope & limitations (honest)

- Gemini and Groq adapters are implemented; OpenAI/Anthropic exist as registry aliases only and return 503 until an adapter is added.
- Client chooses the model — there is no intelligent/automatic routing.
- No retries, fallback, streaming, rate limiting, RBAC, or caching (all deferred to V2).
- Authentication only (identity); authorization/quotas are future work.
- These are intentional V1 boundaries, enabled-for-later by the clean architecture.